RTL Design (Lab Report – Week 1 & 2)

Submitted by:

Mavia Ahmed

Submitted to:

Sir Khallil Rehman

Table of Contents

(RTL Design Lab)

Week#	Date	Lab Tasks
01	15-June-2026 17-June-2026 20-June-2026	1. Implement AND gate using Behavioral, Data Flow and Structural Models 2. Implement OR gate using Behavioral, Data Flow and Structural Models 3. Implement XOR gate using Behavioral, Data Flow and Structural Models 4. Design and run the code for the given schematic on software 5. Implement Half Adder 6. Implement Half Subtractor 7. Implement Full Adder 8. Implement Full Subtractor 9. Design and run the code for the given schematic on software 10. Implement a 4x1 MUX 11. Implement a BCD to 7-Segment Display 12. Implement a 4 input Priority Encoder 13. Implement a 3-to-8 Decoder 14. Implement a 4-to-2 Encoder 15. Implement an 8-bit binary Multiplier
02	22-June-2026 24-June-2026 27-June-2026	1. Implement T-Flipflop in System Verilog. 2. Implement D-Flipflop in System Verilog. 3. Implement SR-Flipflop in System Verilog. 4. Implement JK-Flipflop in System Verilog. 5. Convert a D-Flipflop into a T-Flipflop (Make a T-Flipflop using a D-Flipflop) 6. Convert a T-Flipflop into a D-Flipflop (Make a D-Flipflop using a T-Flipflop) 7. Implement a **4-bit Up/Down Counter (Synchronous)** in System Verilog. 8. Implement a **4-bit Up-Counter (Ripple/Asynchronous)** in System Verilog. 9. Implement a **BCD Counter** in System Verilog. 10. Implement a **Mod N Counter** in System Verilog. 11. Implement a **Clock Frequency Divider** (100Mhz to 25Mhz) in System Verilog. 12. Implement a **Johnson Counter** in System Verilog. 13. Implement a **Ring Counter** in System Verilog.

Table of Contents

(RTL Design Lab)

		14. Implement a **Universal Shift Register (SISO, SIPO, PISO, PIPO)** in System Verilog. 15. Implement a **Bi-Directional Shift Register (SISIO, SIPO)** in System Verilog. 16. Implement an **Edge Detector** in System Verilog. 17. Implement a **32-bit Multiplier (Pipelined)** in System Verilog. 18. Implement a **32-bit ALU** in System Verilog.
03	14-July-2026 15-July-2026 18-July-2026	1. Implement a 1011 **Sequence Detector** using FSM in SystemVerilog. 2. Implement **Traffic Lights** using FSM in SystemVerilog. 3. Implement a **Vending Machine** using FSM in SystemVerilog. 4. Implement an 8-bit **FIFO Buffer** in SystemVerilog. 5. To design and verify a **PS/2 keyboard receiver** in SystemVerilog capable of receiving keyboard scan codes and detecting the keys A, B, and C. 6. To design and verify an **APB-based RAM** module capable of performing read and write operations using the APB protocol.
04		

GitHub Repository Link:

https://github.com/mavia369/USTP-RTL-Design-Labs.git

Mavia Ahmed

Week No. 1

Task 1:

Objective:

- Implement **AND** gate using **Behavioral, Data Flow** and **Structural** Models.

Design Code:

AND Behavioral

```
//Behavioral Model

module and_beh(
    input logic a, b,
    output logic y
);

always @(*) begin
    y = a & b;
end

endmodule
```

AND Data Flow

```
//Data Flow Model

module and_data(
    input logic a,
    input logic b,
    output logic y
);

assign y = a & b;

endmodule
```

AND Structural

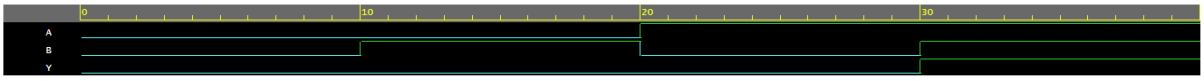
```
// //Structural Model

module and_struct(
    input logic a,b,
    output logic y
);

  and G1(y, a, b);

endmodule
```

Test Bench Code:

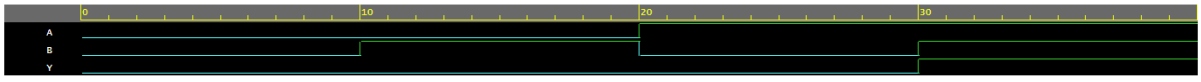
```
//Test Bench
module and_tb;

logic A, B;
logic Y;

and_beh dut(
  .a(A),
  .b(B),
  .y(Y)
);

initial begin
    A=0; B=0; #10;
    A=0; B=1; #10;
    A=1; B=0; #10;
    A=1; B=1; #10;

    $finish;
end

initial begin
  $dumpfile("tb_and.vcd");
  $dumpvars(0, and_tb);
end

endmodule
```

Output:

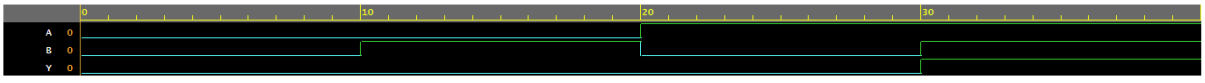
AND Behavioral



AND Data Flow



AND Structural



Mavia Ahmed

Task 2:

Objective:

- Implement **OR** gate using **Behavioral, Data Flow** and **Structural** Models.

Design Code:

OR Behavioral

```
//Behavioral Model
module or_beh(
    input logic a,b,
    output logic y
);

always @(*) begin
    y = a | b;
end

endmodule
```

OR Data Flow

```
//Data Flow Model

module or_data(
    input logic a,b,
    output logic y
);

assign y = a | b;

endmodule
```

OR Structural

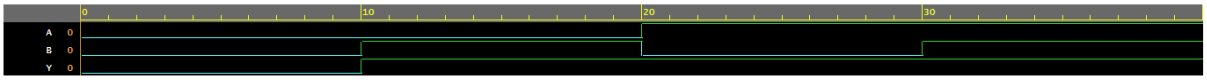
```
//Structural Model

module or_struct(
    input logic a,b,
    output logic y
);

or G1(y, a, b);

endmodule
```

Test Bench Code:

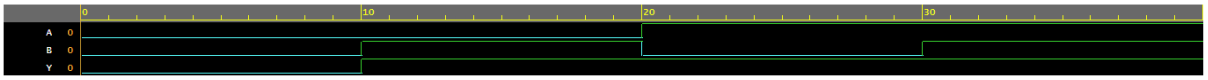
```
//Test Bench
module or_tb;

logic A, B;
logic Y;

or_beh dut(
  .a(A),
  .b(B),
  .y(Y)
);

initial begin
    A=0; B=0; #10;
    A=0; B=1; #10;
    A=1; B=0; #10;
    A=1; B=1; #10;

    $finish;
end

initial begin
  $dumpfile("tb_or.vcd");
  $dumpvars(0, or_tb);
end

endmodule
```

Output:

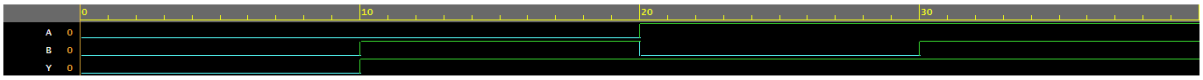
OR Behavioral



OR Data Flow



OR Structural



Mavia Ahmed

Task 3:

Objective:

- Implement **XOR** gate using **Behavioral, Data Flow** and **Structural** Models.

Design Code:

XOR Behavioral

```
//Behavioral Model

module xor_beh(
    input logic a,b,
    output logic y
);

always @(*) begin
    y = a ^ b;
end

endmodule
```

XOR Data Flow

```
//Data Flow Model

module xor_data(
    input logic a,b,
    output logic y
);

assign y = a ^ b;

endmodule
```

XOR Structural

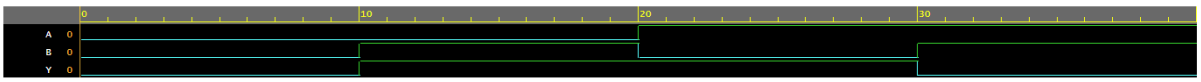
```
//Structural Model

module xor_struct(
    input logic a,b,
    output logic y
);

xor G1(y, a, b);

endmodule
```

Test Bench Code:

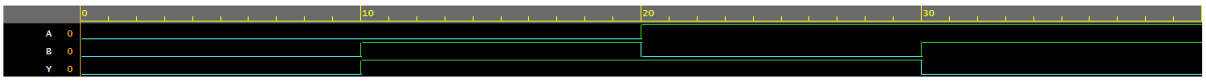
```
//Test Bench
module xor_tb;

logic A, B;
logic Y;

xor_beh dut(
  .a(A),
  .b(B),
  .y(Y)
);

initial begin
    A=0; B=0; #10;
    A=0; B=1; #10;
    A=1; B=0; #10;
    A=1; B=1; #10;

    $finish;
end

initial begin
  $dumpfile("tb_xor.vcd");
  $dumpvars(0, xor_tb);
end

endmodule
```

Output:

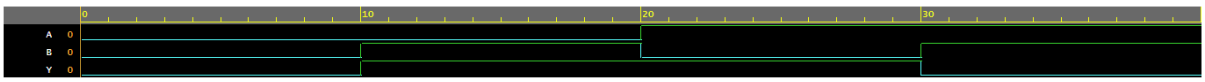
XOR Behavioral



XOR Data Flow



XOR Structural

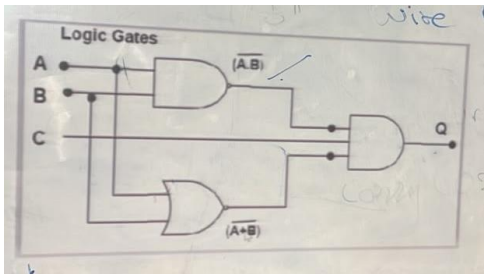


Mavia Ahmed

Task 4:

Objective:

- Design and run the code for the below schematic on software.



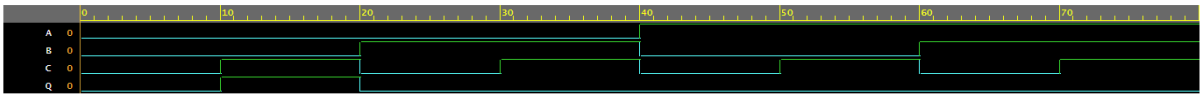
Design Code:

```
module given_schematic(
    input logic a,b,c,
    output logic q
);

always_comb begin
  q = (~(a&b))&(c)&(~(a|b));
end

endmodule
```

Test Bench Code:

```
module given_schematic_tb;

logic A, B, C;
logic Q;

given_schematic dut (
    .a(A),
    .b(B),
    .c(C),
    .q(Q)
);

initial begin
    $dumpfile("given_schematic.vcd");
    $dumpvars(0, given_schematic_tb);

    A=0; B=0; C=0; #10;
    A=0; B=0; C=1; #10;
    A=0; B=1; C=0; #10;
    A=0; B=1; C=1; #10;
    A=1; B=0; C=0; #10;
    A=1; B=0; C=1; #10;
    A=1; B=1; C=0; #10;
    A=1; B=1; C=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 5:

Objective:

Implement **Half Adder.**

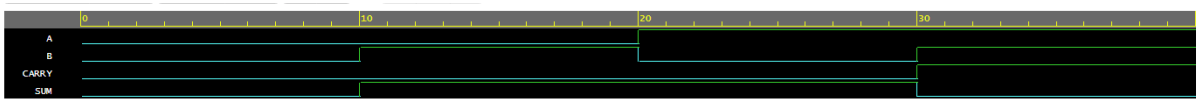
Design Code:

```
module half_adder(
    input logic a, b,
    output logic sum, carry
);

always_comb begin
    sum   = a ^ b;
    carry = a & b;
end

endmodule
```

Test Bench Code:

```
module half_adder_tb;

logic A, B;
logic SUM, CARRY;

half_adder dut(
    .a(A),
    .b(B),
    .sum(SUM),
    .carry(CARRY)
);

initial begin
    $dumpfile("half_adder.vcd");
    $dumpvars(0, half_adder_tb);

    A=0; B=0; #10;
    A=0; B=1; #10;
    A=1; B=0; #10;
    A=1; B=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 6:

Objective:

Implement **Half Subtractor.**

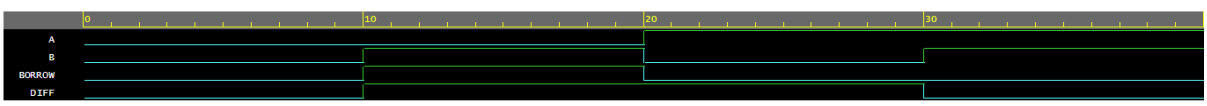
Design Code:

```
module half_subtractor(
    input logic a, b,
    output logic diff, borrow
);

always_comb begin
    diff   = a ^ b;
    borrow = (~a) & b;
end

endmodule
```

Test Bench Code:

```
module half_subtractor_tb;

logic A, B;
logic DIFF, BORROW;

half_subtractor dut(
    .a(A),
    .b(B),
    .diff(DIFF),
    .borrow(BORROW)
);

initial begin
    $dumpfile("half_subtractor.vcd");
    $dumpvars(0, half_subtractor_tb);

    A=0; B=0; #10;
    A=0; B=1; #10;
    A=1; B=0; #10;
    A=1; B=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 7:

Objective:

Implement **Full Adder.**

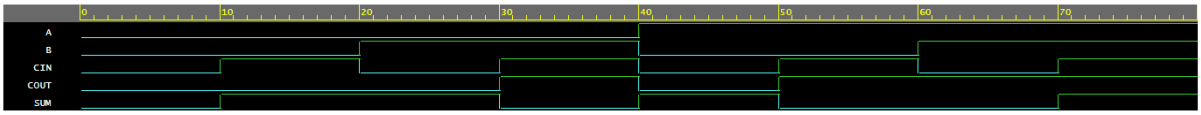
Design Code:

```
module full_adder(
    input logic a, b, cin,
    output logic sum, cout
);

always_comb begin
    sum  = a ^ b ^ cin;
    cout = (a & b) | (b & cin) | (a & cin);
end

endmodule
```

Test Bench Code:

```
module full_adder_tb;

logic A, B, CIN;
logic SUM, COUT;

full_adder dut(
    .a(A),
    .b(B),
    .cin(CIN),
    .sum(SUM),
    .cout(COUT)
);

initial begin
    $dumpfile("full_adder.vcd");
    $dumpvars(0, full_adder_tb);

    A=0; B=0; CIN=0; #10;
    A=0; B=0; CIN=1; #10;
    A=0; B=1; CIN=0; #10;
    A=0; B=1; CIN=1; #10;
    A=1; B=0; CIN=0; #10;
    A=1; B=0; CIN=1; #10;
    A=1; B=1; CIN=0; #10;
    A=1; B=1; CIN=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 8:

Objective:

Implement **Full Subtractor.**

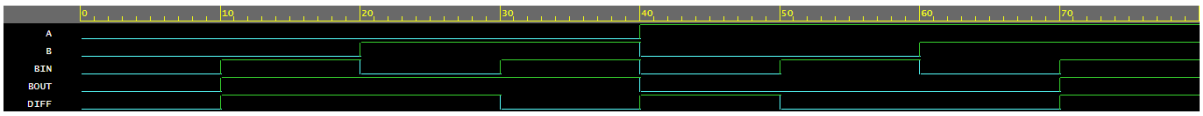
Design Code:

```
module full_subtractor(
    input logic a, b, bin,
    output logic diff, bout
);

always_comb begin
    diff = a ^ b ^ bin;
    bout = (~a & b) | (~a & bin) | (b & bin);
end

endmodule
```

Test Bench Code:

```
module full_subtractor_tb;

logic A, B, BIN;
logic DIFF, BOUT;

full_subtractor dut(
    .a(A),
    .b(B),
    .bin(BIN),
    .diff(DIFF),
    .bout(BOUT)
);

initial begin
    $dumpfile("full_subtractor.vcd");
    $dumpvars(0, full_subtractor_tb);

    A=0; B=0; BIN=0; #10;
    A=0; B=0; BIN=1; #10;
    A=0; B=1; BIN=0; #10;
    A=0; B=1; BIN=1; #10;
    A=1; B=0; BIN=0; #10;
    A=1; B=0; BIN=1; #10;
    A=1; B=1; BIN=0; #10;
    A=1; B=1; BIN=1; #10;

    $finish;
end

endmodule
```

Output:

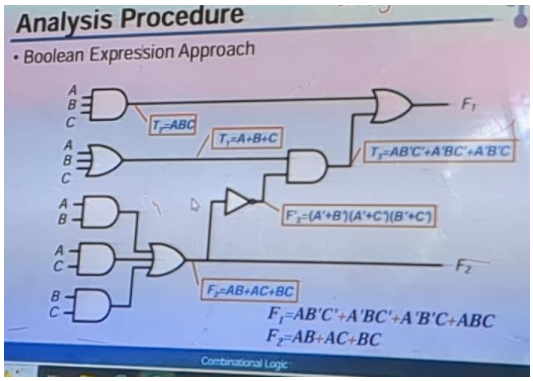


Mavia Ahmed

Task 9:

Objective:

- Design and run the code for the below schematic on software.



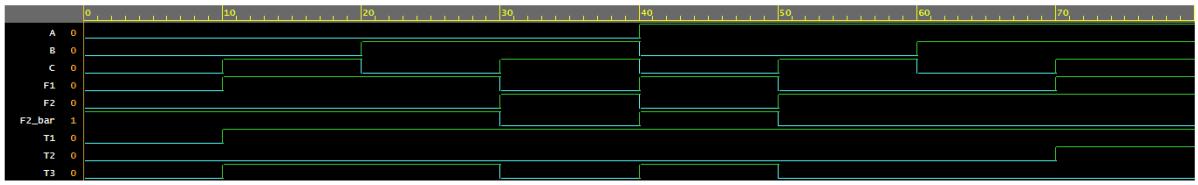
Design Code:

```
module circuit(
    input logic A, B, C,
    output logic F1, F2
);

logic T1, T2, F2_bar, T3;

always_comb begin
    T1     = A | B | C;
    T2     = A & B & C;

    F2     = (A & B) | (A & C) | (B & C);

    F2_bar = (~A | ~B) &
             (~A | ~C) &
             (~B | ~C);

    T3     = (A & ~B & ~C) |
             (~A & B & ~C) |
             (~A & ~B & C);

    F1     = (A & ~B & ~C) |
             (~A & B & ~C) |
             (~A & ~B & C) |
             (A & B & C);
end

endmodule
```

Test Bench Code:

```
module circuit_tb;

logic A, B, C;
logic F1, F2;

circuit dut(
    .A(A),
    .B(B),
    .C(C),
    .F1(F1),
    .F2(F2)
);

initial begin
    $dumpfile("circuit.vcd");
    $dumpvars(0, circuit_tb);

    A=0; B=0; C=0; #10;
    A=0; B=0; C=1; #10;
    A=0; B=1; C=0; #10;
    A=0; B=1; C=1; #10;
    A=1; B=0; C=0; #10;
    A=1; B=0; C=1; #10;
    A=1; B=1; C=0; #10;
    A=1; B=1; C=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 10:

Objective:

Implement a 4x1 **MUX**.

Design Code:

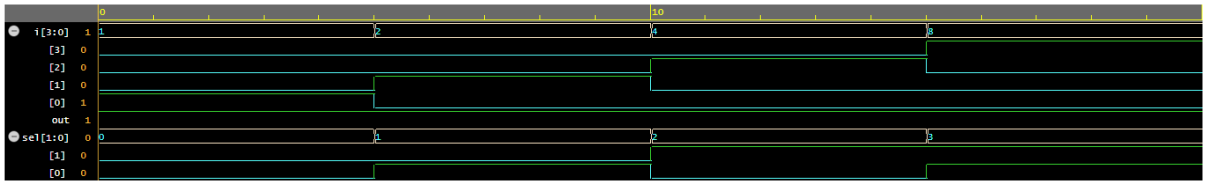
```
module mux4to1(
  input logic [3:0]i,
  input logic [1:0] sel,
  output logic out
);

  always_comb begin
    case(sel)
      2'b00:out=i[0];
      2'b01:out=i[1];
      2'b10:out=i[2];
      2'b11:out=i[3];
      default: out = 0;
    endcase
  end

endmodule
```

Test Bench Code:

```
module tb_mux4to1;

  logic [3:0] i;
  logic [1:0] sel;
  logic       out;

  mux4to1 dut (
    .i(i),
    .sel(sel),
    .out(out)
  );

  initial begin
    $dumpfile("mux4to1_tb.vcd");
    $dumpvars(0, tb_mux4to1);

    i = 4'b0001; sel = 2'b00; #5;
    i = 4'b0010; sel = 2'b01; #5;
    i = 4'b0100; sel = 2'b10; #5;
    i = 4'b1000; sel = 2'b11; #5;
    $finish;
  end

endmodule
```

Output:

Mavia Ahmed



Mavia Ahmed

Task 11:

Objective:

Implement a **BCD** to **7-Segment Display**.

Design Code:

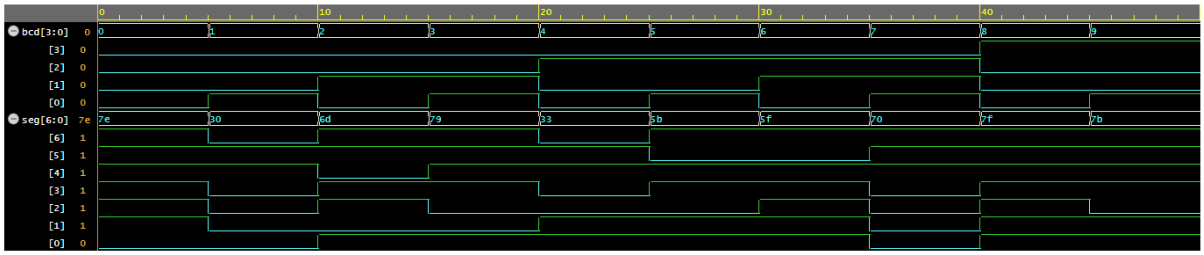
```
module bcd_to_7seg(
  input  logic [3:0] bcd,
  output logic [6:0] seg
);

  always_comb begin
    case (bcd)
      4'b0000: seg = 7'b1111110;
      4'b0001: seg = 7'b0110000;
      4'b0010: seg = 7'b1101101;
      4'b0011: seg = 7'b1111001;
      4'b0100: seg = 7'b0110011;
      4'b0101: seg = 7'b1011011;
      4'b0110: seg = 7'b1011111;
      4'b0111: seg = 7'b1110000;
      4'b1000: seg = 7'b1111111;
      4'b1001: seg = 7'b1111011;
      default: seg = 7'b0000000;
    endcase
  end

endmodule
```

Test Bench Code:

```
module tb_bcd_to_7seg;

  logic [3:0] bcd;
  logic [6:0] seg;

  bcd_to_7seg dut(
    .bcd(bcd),
    .seg(seg)
  );

  initial begin
    bcd = 4'b0000; #5;
    bcd = 4'b0001; #5;
    bcd = 4'b0010; #5;
    bcd = 4'b0011; #5;
    bcd = 4'b0100; #5;
    bcd = 4'b0101; #5;
    bcd = 4'b0110; #5;
    bcd = 4'b0111; #5;
    bcd = 4'b1000; #5;
    bcd = 4'b1001; #5;

    $finish;
  end

   initial begin
    $dumpfile("bcd_to_7seg_tb.vcd");
    $dumpvars(0, tb_bcd_to_7seg);
  end

endmodule
```

Mavia Ahmed

Output:



Mavia Ahmed

Task 12:

Objective:

Implement a 4 input **Priority Encoder**.

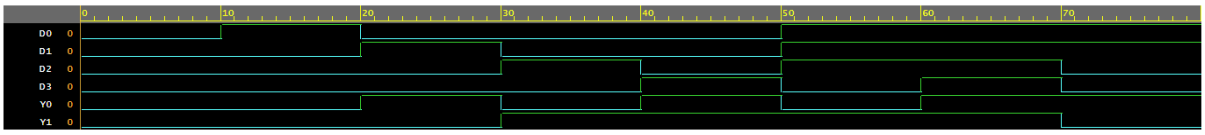
Design Code:

```
module priority_encoder_4to2(
    input  logic D3, D2, D1, D0,
    output logic Y1, Y0
);

always_comb begin
    if (D3) begin
        Y1 = 1;
        Y0 = 1;
    end
    else if (D2) begin
        Y1 = 1;
        Y0 = 0;
    end
    else if (D1) begin
        Y1 = 0;
        Y0 = 1;
    end
    else begin
        Y1 = 0;
        Y0 = 0;
    end
end

endmodule
```

Test Bench Code:

```
module priority_encoder_4to2_tb;

logic D3, D2, D1, D0;
logic Y1, Y0;

priority_encoder_4to2 dut(
    .D3(D3),
    .D2(D2),
    .D1(D1),
    .D0(D0),
    .Y1(Y1),
    .Y0(Y0)
);

initial begin
    $dumpfile("priority_encoder.vcd");
    $dumpvars(0, priority_encoder_4to2_tb);

    D3=0; D2=0; D1=0; D0=0; #10;
    D3=0; D2=0; D1=0; D0=1; #10;
    D3=0; D2=0; D1=1; D0=0; #10;
    D3=0; D2=1; D1=0; D0=0; #10;
    D3=1; D2=0; D1=0; D0=0; #10;

    // Priority checks
    D3=0; D2=1; D1=1; D0=1; #10;
    D3=1; D2=1; D1=1; D0=1; #10;
    D3=0; D2=0; D1=1; D0=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 13:

Objective:

Implement a 3-to-8 **Decoder**.

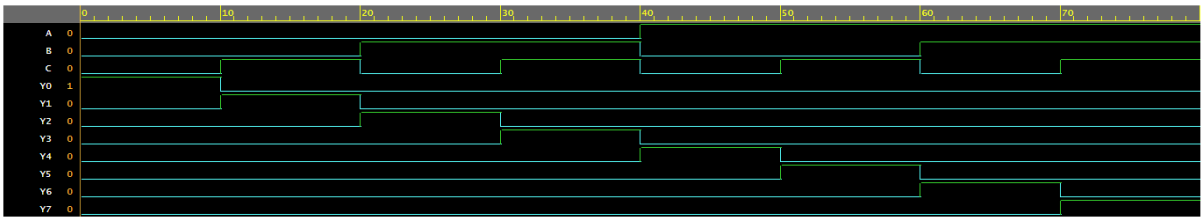
Design Code:

```
module decoder_3to8(
    input logic A, B, C,
    output logic Y0, Y1, Y2, Y3,
                 Y4, Y5, Y6, Y7
);

always_comb begin
    Y0=0; Y1=0; Y2=0; Y3=0;
    Y4=0; Y5=0; Y6=0; Y7=0;

    case ({A,B,C})
        3'b000: Y0 = 1;
        3'b001: Y1 = 1;
        3'b010: Y2 = 1;
        3'b011: Y3 = 1;
        3'b100: Y4 = 1;
        3'b101: Y5 = 1;
        3'b110: Y6 = 1;
        3'b111: Y7 = 1;
    endcase
end

endmodule
```

Test Bench Code:

```
module decoder_3to8_tb;

logic A, B, C;
logic Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7;

decoder_3to8 dut(
    .A(A), .B(B), .C(C),
    .Y0(Y0), .Y1(Y1), .Y2(Y2), .Y3(Y3),
    .Y4(Y4), .Y5(Y5), .Y6(Y6), .Y7(Y7)
);

initial begin
    $dumpfile("decoder_3to8.vcd");
    $dumpvars(0, decoder_3to8_tb);

    A=0; B=0; C=0; #10;
    A=0; B=0; C=1; #10;
    A=0; B=1; C=0; #10;
    A=0; B=1; C=1; #10;
    A=1; B=0; C=0; #10;
    A=1; B=0; C=1; #10;
    A=1; B=1; C=0; #10;
    A=1; B=1; C=1; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 14:

Objective:

Implement a 4-to-2 **Encoder**.

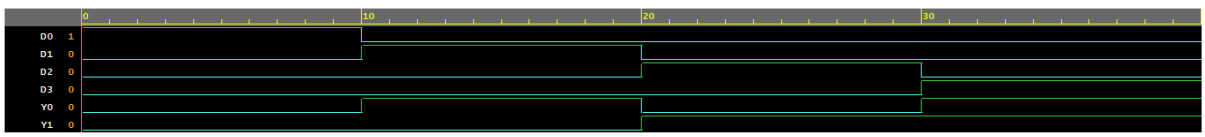
Design Code:

```
module encoder_4to2(
    input logic D3, D2, D1, D0,
    output logic Y1, Y0
);

always_comb begin
    Y1 = D2 | D3;
    Y0 = D1 | D3;
end

endmodule
```

Test Bench Code:

```
module encoder_4to2_tb;

logic D3, D2, D1, D0;
logic Y1, Y0;

encoder_4to2 dut(
    .D3(D3),
    .D2(D2),
    .D1(D1),
    .D0(D0),
    .Y1(Y1),
    .Y0(Y0)
);

initial begin
    $dumpfile("encoder_4to2.vcd");
    $dumpvars(0, encoder_4to2_tb);

    D3=0; D2=0; D1=0; D0=1; #10;
    D3=0; D2=0; D1=1; D0=0; #10;
    D3=0; D2=1; D1=0; D0=0; #10;
    D3=1; D2=0; D1=0; D0=0; #10;

    $finish;
end

endmodule
```

Output:



Mavia Ahmed

Task 15:

Objective:

Implement an **8-bit binary Multiplier**.

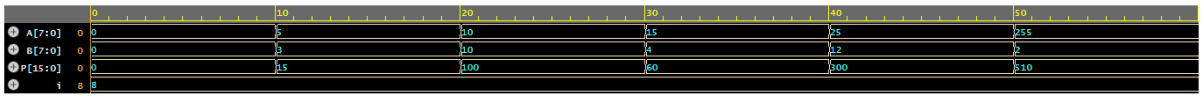
Design Code:

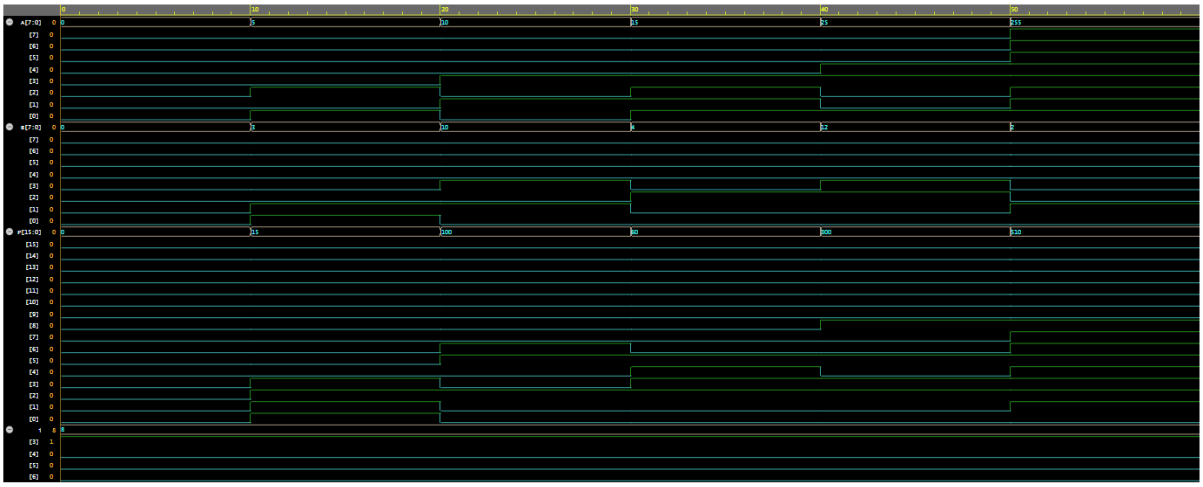
```
module multiplier_8bit(
    input  logic [7:0] A,
    input  logic [7:0] B,
    output logic [15:0] P
);

integer i;

always_comb begin
    P = 16'd0;

    for (i = 0; i < 8; i = i + 1) begin
        if (B[i])
            P = P + (A << i);
    end
end

endmodule
```

Test Bench Code:

```
module multiplier_8bit_tb;

logic [7:0] A, B;
logic [15:0] P;

multiplier_8bit dut(
    .A(A),
    .B(B),
    .P(P)
);

initial begin
    $dumpfile("multiplier_8bit.vcd");
    $dumpvars(0, multiplier_8bit_tb);

    A = 8'd0;   B = 8'd0;   #10;
    A = 8'd5;   B = 8'd3;   #10;
    A = 8'd10;  B = 8'd10;  #10;
    A = 8'd15;  B = 8'd4;   #10;
    A = 8'd25;  B = 8'd12;  #10;
    A = 8'd255; B = 8'd2;   #10;

    $finish;
end

endmodule
```

Output:


0
10
20
30
40
50
A[7:0] 0 0 5 10 15 25 255
B[7:0] 0 0 3 10 4 12 2
P[15:0] 0 0 15 100 60 300 510
i 8 8



Mavia Ahmed

Week No. 2

Task 1:

Objective:

Implement T-Flipflop in System Verilog.

Design Code:

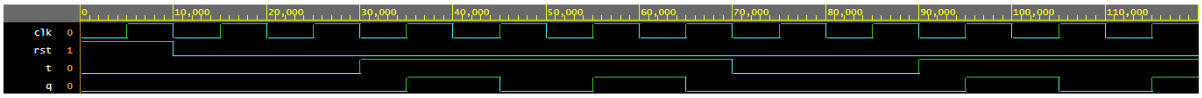
```
module t_flipflop (
    input  logic clk,
    input  logic rst,
    input  logic t,
    output logic q
);

always_ff @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else if (t)
        q <= ~q;
    else
        q <= q;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module t_flipflop_tb;

logic clk;
logic rst;
logic t;
logic q;

t_flipflop uut (
    .clk(clk),
    .rst(rst),
    .t(t),
    .q(q)
);
```

```
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    t   = 0;

    #10;
    rst = 0;

    #10;
    t = 0;

    #10;
    t = 1;

    #40;

    t = 0;
    #20;

    t = 1;
    #30;

    $finish;
end

initial begin
    $dumpfile("t_flipflop.vcd");
    $dumpvars(0, t_flipflop_tb);
end

endmodule
```

Output:



Task 2:

Objective:

Implement D-Flipflop in System Verilog.

Design Code:

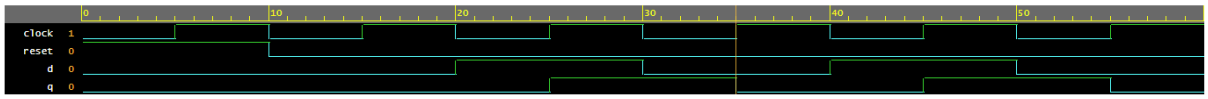
```
module d_flipflop(
    input logic clock, reset,
    input logic d,
    output logic q
);

always_ff @(posedge clock or posedge reset) begin
  if (reset)
    q <= 0;
  else
    q <= d;
end

endmodule
```

Test Bench Code:

```
module tb_d_flipflop;

logic clock, reset, d;
logic q;

d_flipflop dut(
  .clock(clock),
  .reset(reset),
  .d(d),
  .q(q)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    d = 0;

    #10;
    reset = 0;

    #10 d = 1;
    #10 d = 0;
    #10 d = 1;
```

```
    #10 d = 0;

    #10;
    $finish;
end

initial begin
  $dumpfile("d_flipflop.vcd");
  $dumpvars(0,tb_d_flipflop);
end

endmodule
```

Output:



Mavia Ahmed

Task 3:

Objective:

Implement SR-Flipflop in System Verilog.

Design Code:

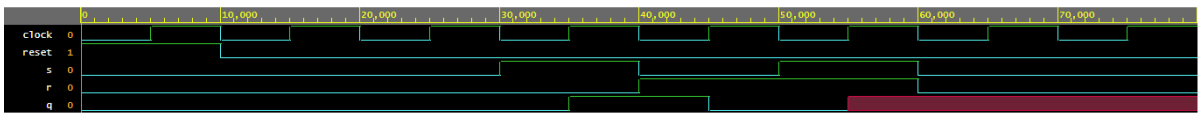
```
module sr_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic s,
    input  logic r,
    output logic q
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else begin
        case ({s, r})
            2'b00: q <= q;
            2'b01: q <= 1'b0;
            2'b10: q <= 1'b1;
            2'b11: q <= 1'bx;
        endcase
    end
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_sr_flipflop;

logic clock;
logic reset;
logic s;
logic r;
logic q;

sr_flipflop dut (
    .clock(clock),
    .reset(reset),
    .s(s),
    .r(r),
    .q(q)
);
```

```
always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    s = 0;
    r = 0;

    #10;
    reset = 0;

    #10;
    s = 0;
    r = 0;

    #10;
    s = 1;
    r = 0;

    #10;
    s = 0;
    r = 1;

    #10;
    s = 1;
    r = 1;

    #10;

    s = 0;
    r = 0;

    #20;

    $finish;
end

initial begin
    $dumpfile("sr_flipflop.vcd");
    $dumpvars(0, tb_sr_flipflop);
end

endmodule
```

Output:



Mavia Ahmed

Task 4:

Objective:

Implement JK-Flipflop in System Verilog.

Design Code:

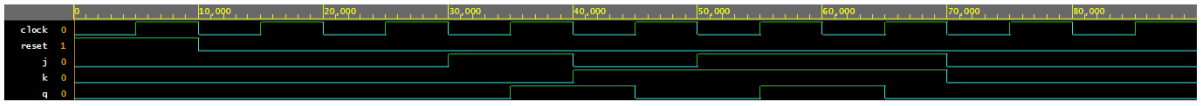
```
module jk_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic j,
    input  logic k,
    output logic q
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else begin
        case ({j, k})
            2'b00: q <= q;
            2'b01: q <= 1'b0;
            2'b10: q <= 1'b1;
            2'b11: q <= ~q;
        endcase
    end
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_jk_flipflop;

logic clock;
logic reset;
logic j;
logic k;
logic q;

jk_flipflop dut (
    .clock(clock),
    .reset(reset),
    .j(j),
    .k(k),
    .q(q)
);
```

```
always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    j = 0;
    k = 0;

    #10;
    reset = 0;

    #10;
    j = 0;
    k = 0;

    #10;
    j = 1;
    k = 0;

    #10;
    j = 0;
    k = 1;


    #10;
    j = 1;
    k = 1;

    #20;

    j = 0;
    k = 0;

    #20;

    $finish;
end

initial begin
    $dumpfile("jk_flipflop.vcd");
    $dumpvars(0, tb_jk_flipflop);
end

endmodule
```

Output:



Mavia Ahmed

Task 5:

Objective:

Convert a D-Flipflop into a T-Flipflop (Make a T-Flipflop using a D-Flipflop)

(Hint: Characteristic equation).

Design Code:

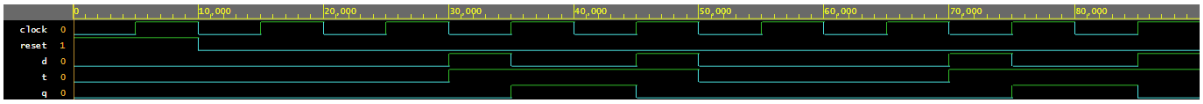
```
module d_to_t_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic t,
    output logic q
);

logic d;

assign d = t ^ q;

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else
        q <= d;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_d_to_t_flipflop;

logic clock;
logic reset;
logic t;
logic q;

d_to_t_flipflop dut(
    .clock(clock),
    .reset(reset),
    .t(t),
    .q(q)
);

always #5 clock = ~clock;
```

```
initial begin
    clock = 0;
    reset = 1;
    t = 0;

    #10;
    reset = 0;

    #10 t = 0;

    #10 t = 1;

    #20;

    t = 0;
    #20;

    t = 1;
    #20;

    $finish;
end

initial begin
    $dumpfile("d_to_t.vcd");
    $dumpvars(0, tb_d_to_t_flipflop);
end

endmodule
```

Output:



Mavia Ahmed

Task 6:

Objective:

Convert a T-Flipflop into a D-Flipflop (Make a D-Flipflop using a T-Flipflop)

(Hint: Characteristic equation).

Design Code:

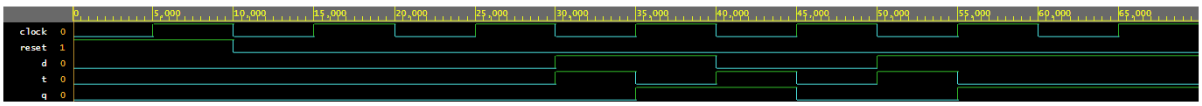
```
module t_to_d_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic d,
    output logic q
);

logic t;

assign t = d ^ q;

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else if (t)
        q <= ~q;
    else
        q <= q;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_t_to_d_flipflop;

logic clock;
logic reset;
logic d;
logic q;

t_to_d_flipflop dut(
    .clock(clock),
    .reset(reset),
    .d(d),
    .q(q)
);
```

```
always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    d = 0;

    #10;
    reset = 0;

    #10 d = 0;
    #10 d = 1;
    #10 d = 0;
    #10 d = 1;
    #10 d = 1;
    #10 d = 0;

    $finish;
end

initial begin
    $dumpfile("t_to_d.vcd");
    $dumpvars(0, tb_t_to_d_flipflop);
end

endmodule
```

Output:



Mavia Ahmed

Task 7:

Objective:

Implement a 4-bit **Up/Down Counter (Synchronous)** in System Verilog.

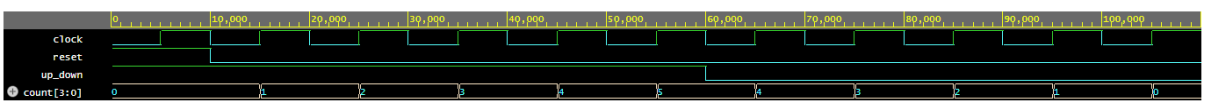
Design Code:

```
module up_down_counter_synchronous (
    input clock,
    input reset,
    input up_down,
    output reg [3:0] count
);

  always @(posedge clock or posedge reset) begin
  if(reset)
    count <= 4'b0000;
  else if (up_down)
    count <= count + 1;
  else
    count <= count -1;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_up_down_counter_synchronous;

reg clock;
reg reset;
reg up_down;
wire [3:0] count;

up_down_counter_synchronous uut (
  .clock(clock),
  .reset(reset),
  .up_down(up_down),
  .count(count)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    up_down = 1;

    #10 reset = 0;

    #50;

    up_down = 0;
    #50;

    $finish;
end

initial begin
    $dumpfile("up_down_counter_synchronous.vcd");
    $dumpvars(0, tb_up_down_counter_synchronous);
end

endmodule
```

Output:



Mavia Ahmed

Task 8:

Objective:

Implement a 4-bit **Up-Counter (Ripple/Asynchronous)** in System Verilog.

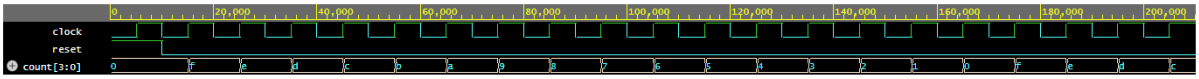
Design Code:

```
module async_up_counter (
    input clock,
    input reset,
    output reg [3:0] count
);

always @(posedge clock or posedge reset) begin
  if (reset)
        count[0] <= 0;
  else
        count[0] <= ~count[0];
end

always @(posedge count[0] or posedge reset) begin
  if (reset)
        count[1] <= 0;
  else
        count[1] <= ~count[1];
end

always @(posedge count[1] or posedge reset) begin
  if (reset)
        count[2] <= 0;
  else
        count[2] <= ~count[2];
end

always @(posedge count[2] or posedge reset) begin
  if (reset)
        count[3] <= 0;
  else
        count[3] <= ~count[3];
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_async_up_counter;

reg clock;
reg reset;
wire [3:0] count;

async_up_counter uut (
  .clock(clock),
  .reset(reset),
  .count(count)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;

    #10 reset = 0;

    #200;

    $finish;
end

initial begin
  $dumpfile("async_up_counter.vcd");
  $dumpvars(0, tb_async_up_counter);
end

endmodule
```

Output:



Mavia Ahmed

Task 9:

Objective:

Implement a **BCD Counter** in System Verilog.

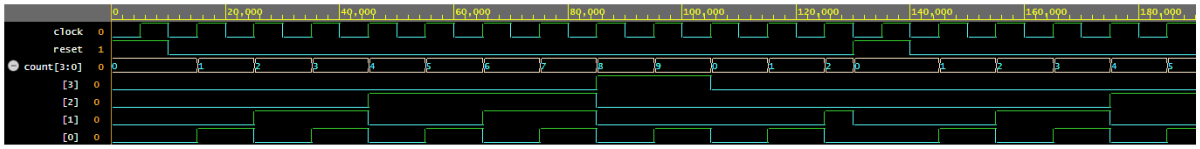
Design Code:

```
module bcd_counter(
    input  logic clock,
    input  logic reset,
    output logic [3:0] count
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        count <= 4'd0;
    else if (count == 4'd9)
        count <= 4'd0;
    else
        count <= count + 1'b1;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_bcd_counter;

logic clock;
logic reset;
logic [3:0] count;

bcd_counter dut (
    .clock(clock),
    .reset(reset),
    .count(count)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;

    #10;
    reset = 0;

    #120;

    reset = 1;
    #10;
    reset = 0;

    #50;

    $finish;
end

initial begin
    $dumpfile("bcd_counter.vcd");
    $dumpvars(0, tb_bcd_counter);
end

endmodule
```

Output:


0
20,000
40,000
60,000
80,000
100,000
120,000
140,000
160,000
180,000
clock 0
reset 1
count[3:0] 0
0 1 2 3 4 5 6 7 8 9 0 1 2 0 1 2 3 4 5
[3] 0
[2] 0
[1] 0
[0] 0

Mavia Ahmed

Task 10:

Objective:

Implement a **Mod N Counter** in System Verilog.

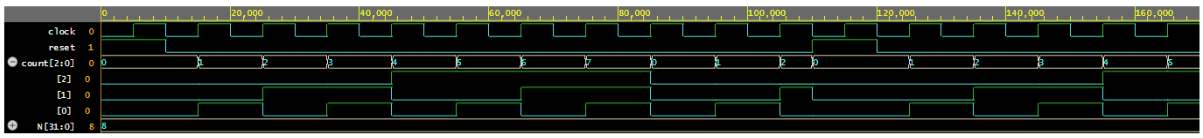
Design Code:

```
module mod_n_counter #(
    parameter N = 10
)(
    input  logic clock,
    input  logic reset,
    output logic [$clog2(N)-1:0] count
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        count <= 0;
    else if (count == N-1)
        count <= 0;
    else
        count <= count + 1'b1;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_mod_n_counter;

parameter N = 8;

logic clock;
logic reset;
logic [$clog2(N)-1:0] count;

mod_n_counter #(.N(N)) dut (
    .clock(clock),
    .reset(reset),
    .count(count)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;

    #10;
    reset = 0;

    #100;

    reset = 1;
    #10;
    reset = 0;

    #50;

    $finish;
end

initial begin
    $dumpfile("mod_n_counter.vcd");
    $dumpvars(0, tb_mod_n_counter);
end

endmodule
```

Output:


0
20,000
40,000
60,000
80,000
100,000
120,000
140,000
160,000
clock 0
reset 1
count[2:0] 0
0 1 2 3 4 5 6 7 0 1 2 0 1 2 3 4 5
[2] 0
[1] 0
[0] 0
N[31:0] 8

Mavia Ahmed

Task 11:

Objective:

Implement a **Clock Frequency Divider** (100Mhz to 25Mhz) in System Verilog.

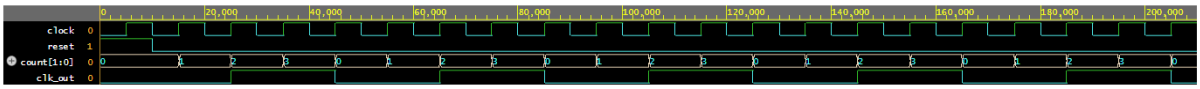
Design Code:

```
module clock_divider(
    input  logic clock,
    input  logic reset,
    output logic clk_out
);

logic [1:0] count;

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 1'b1;
end

assign clk_out = count[1];

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_clock_divider;

logic clock;
logic reset;
logic clk_out;

clock_divider dut(
    .clock(clock),
    .reset(reset),
    .clk_out(clk_out)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;

    #10;
    reset = 0;

    #200;

    $finish;
end

initial begin
    $dumpfile("clock_divider.vcd");
    $dumpvars(0, tb_clock_divider);
end

endmodule
```

Output:


0
20,000
40,000
60,000
80,000
100,000
120,000
140,000
160,000
180,000
200,000
clock
reset
count[1:0]
clk_out

Mavia Ahmed

Task 12:

Objective:

Implement a **Johnson Counter** in System Verilog.

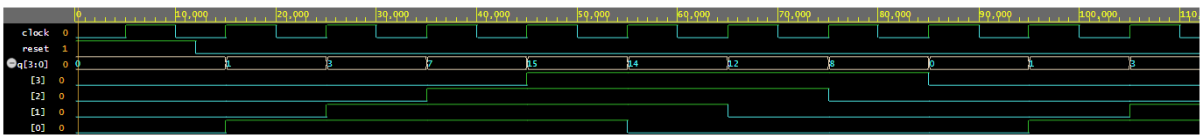
Design Code:

```
module johnson_counter (
    input  logic clock,
    input  logic reset,
    output logic [3:0] q
);

  always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= {q[2:0], ~q[3]};
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_johnson_counter;

logic clock;
logic reset;
logic [3:0] q;

johnson_counter dut (
  .clock(clock),
  .reset(reset),
    .q(q)
);

initial begin
    clock = 0;
    forever #5 clock = ~clock;
end

initial begin
    reset = 1;

    #12;
    reset = 0;

    #100;

    $finish;
end

initial begin
    $dumpfile("johnson_counter.vcd");
    $dumpvars(0, tb_johnson_counter);
end

endmodule
```

Output:



Mavia Ahmed

Task 13:

Objective:

Implement a **Ring Counter** in System Verilog.

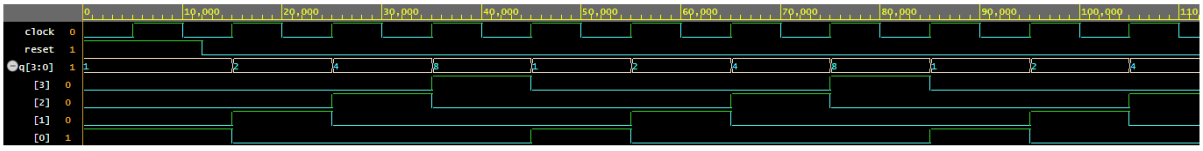
Design Code:

```
module ring_counter (
    input  logic clock,
    input  logic reset,
    output logic [3:0] q
);

  always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 4'b0001;
    else
        q <= {q[2:0], q[3]};
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_ring_counter;

logic clock;
logic reset;
logic [3:0] q;

ring_counter dut (
  .clock(clock),
  .reset(reset),
    .q(q)
);

initial begin
    clock = 0;
    forever #5 clock = ~clock;
end

initial begin
    reset = 1;

    #12;
    reset = 0;

    #100;

    $finish;
end

initial begin
    $dumpfile("ring_counter.vcd");
    $dumpvars(0, tb_ring_counter);
end

endmodule
```

Output:


0
10,000
20,000
30,000
40,000
50,000
60,000
70,000
80,000
90,000
100,000
110
clock 0
reset 1
q[3:0] 1
1
2
4
8
1
2
4
8
1
2
4
[3] 0
[2] 0
[1] 0
[0] 1

Mavia Ahmed

Task 14:

Objective:

Implement a **Universal Shift Register (SISO, SIPO, PISO, PIPO)** in System Verilog.

Design Code:

```
module universal_shift_register(
    input  logic clock,
    input  logic reset,

    input  logic [1:0] mode,

    input  logic serial_left_in,
    input  logic serial_right_in,

    input  logic [3:0] parallel_in,

    output logic [3:0] q,

    output logic serial_left_out,
    output logic serial_right_out
);

always_ff @(posedge clock or posedge reset) begin

    if (reset)
        q <= 4'b0000;

    else begin
        case (mode)

            2'b00:
                q <= q;

            2'b01:
                q <= {serial_left_in, q[3:1]};

            2'b10:
                q <= {q[2:0], serial_right_in};

            2'b11:
                q <= parallel_in;

            default:
                q <= q;

        endcase
    end
```

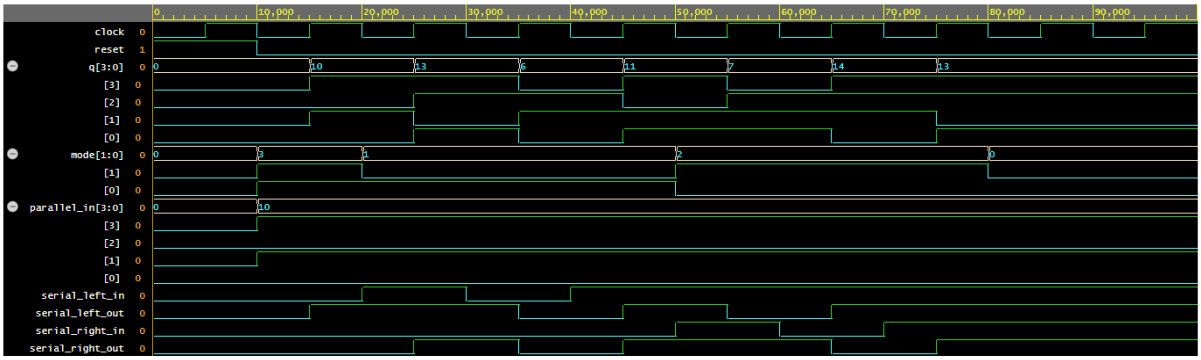
```
end

assign serial_left_out  = q[3];
assign serial_right_out = q[0];

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_universal_shift_register;

logic clock;
logic reset;

logic [1:0] mode;

logic serial_left_in;
logic serial_right_in;

logic [3:0] parallel_in;

logic [3:0] q;

logic serial_left_out;
logic serial_right_out;

universal_shift_register dut(

    .clock(clock),
    .reset(reset),

    .mode(mode),

    .serial_left_in(serial_left_in),
    .serial_right_in(serial_right_in),

    .parallel_in(parallel_in),

    .q(q),

    .serial_left_out(serial_left_out),
    .serial_right_out(serial_right_out)

);

always #5 clock = ~clock;

initial begin

    clock = 0;
```

```
44
45      reset = 1;
46      mode = 2'b00;
47
48      serial_left_in = 0;
49      serial_right_in = 0;
50
51      parallel_in = 4'b0000;
52
53      #10;
54      reset = 0;
55
56      mode = 2'b11;
57      parallel_in = 4'b1010;
58      #10;
59
60      mode = 2'b01;
61
62      serial_left_in = 1; #10;
63      serial_left_in = 0; #10;
64      serial_left_in = 1; #10;
65
66      mode = 2'b10;
67
68      serial_right_in = 1; #10;
69      serial_right_in = 0; #10;
70      serial_right_in = 1; #10;
71
72      mode = 2'b00;
73      #20;
74
75      $finish;
76
77  end
78
79  initial begin
80      $dumpfile("universal_shift_register.vcd");
81      $dumpvars(0, tb_universal_shift_register);
82  end
83
84  endmodule
```

Output:



Mavia Ahmed

Task 15:

Objective:

Implement a **Bi-Directional Shift Register (SISIO, SIPO)** in System Verilog.

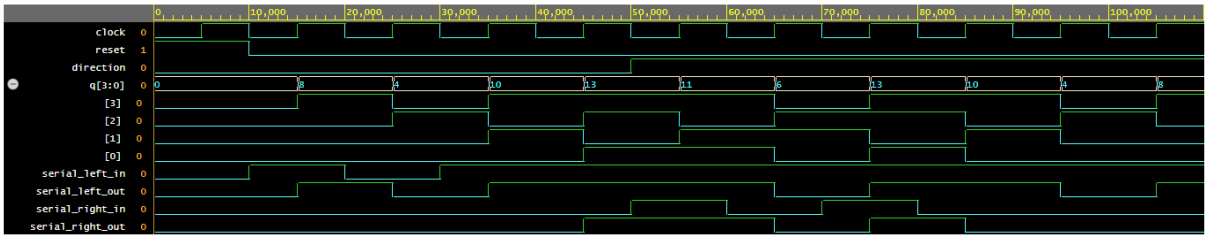
Design Code:

```
module bidirectional_shift_register(
    input  logic clock,
    input  logic reset,
    input  logic direction,
    input  logic serial_left_in,
    input  logic serial_right_in,
    output logic [3:0] q,
    output logic serial_left_out,
    output logic serial_right_out
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else begin
        if (direction == 1'b0)
            q <= {serial_left_in, q[3:1]};
        else
            q <= {q[2:0], serial_right_in};
    end
end

assign serial_left_out  = q[3];
assign serial_right_out = q[0];

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_bidirectional_shift_register;

logic clock;
logic reset;
logic direction;
logic serial_left_in;
logic serial_right_in;

logic [3:0] q;
logic serial_left_out;
logic serial_right_out;

bidirectional_shift_register dut(
    .clock(clock),
    .reset(reset),
    .direction(direction),
    .serial_left_in(serial_left_in),
    .serial_right_in(serial_right_in),
    .q(q),
    .serial_left_out(serial_left_out),
    .serial_right_out(serial_right_out)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    direction = 0;
    serial_left_in = 0;
    serial_right_in = 0;

    #10;
    reset = 0;

    direction = 0;
    serial_left_in = 1; #10;
    serial_left_in = 0; #10;
    serial_left_in = 1; #10;
    serial_left_in = 1; #10;

    direction = 1;
    serial_right_in = 1; #10;
    serial_right_in = 0; #10;
    serial_right_in = 1; #10;
    serial_right_in = 0; #10;

    #20;

    $finish;
end

initial begin
    $dumpfile("bidirectional_shift_register.vcd");
    $dumpvars(0, tb_bidirectional_shift_register);
end

endmodule
```

Output:


0
10,000
20,000
30,000
40,000
50,000
60,000
70,000
80,000
90,000
100,000
clock 0
reset 1
direction 0
q[3:0] 0
0
8
4
10
13
11
6
13
10
4
8
[3] 0
[2] 0
[1] 0
[0] 0
serial_left_in 0
serial_left_out 0
serial_right_in 0
serial_right_out 0

Mavia Ahmed

Task 16:

Objective:

Implement an **Edge Detector** in System Verilog.

Design Code:

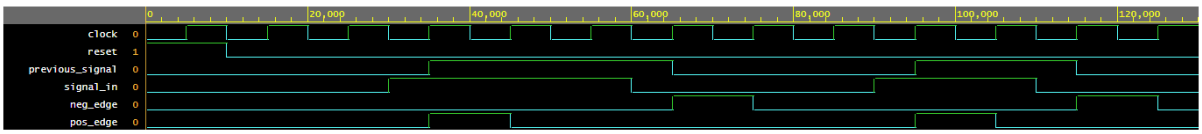
```
module edge_detector(
    input  logic clock,
    input  logic reset,
    input  logic signal_in,
    output logic pos_edge,
    output logic neg_edge
);

logic previous_signal;

always_ff @(posedge clock or posedge reset) begin
    if (reset) begin
        previous_signal <= 1'b0;
        pos_edge        <= 1'b0;
        neg_edge        <= 1'b0;
    end
    else begin
        pos_edge <= signal_in & ~previous_signal;
        neg_edge <= ~signal_in & previous_signal;
        previous_signal <= signal_in;
    end
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_edge_detector;

logic clock;
logic reset;
logic signal_in;
logic pos_edge;
logic neg_edge;

edge_detector dut(
    .clock(clock),
    .reset(reset),
    .signal_in(signal_in),
    .pos_edge(pos_edge),
    .neg_edge(neg_edge)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    signal_in = 0;

    #10;
    reset = 0;

    #10 signal_in = 0;
    #10 signal_in = 1;
    #20 signal_in = 1;
    #10 signal_in = 0;
    #20 signal_in = 0;
    #10 signal_in = 1;
    #20 signal_in = 0;

    #20;

    $finish;
end
```

```
initial begin
    $dumpfile("edge_detector.vcd");
    $dumpvars(0, tb_edge_detector);
end

endmodule
```

Output:



Mavia Ahmed

Task 17:

Objective:

Implement a **32-bit Multiplier (Pipelined)** in System Verilog.

Design Code:

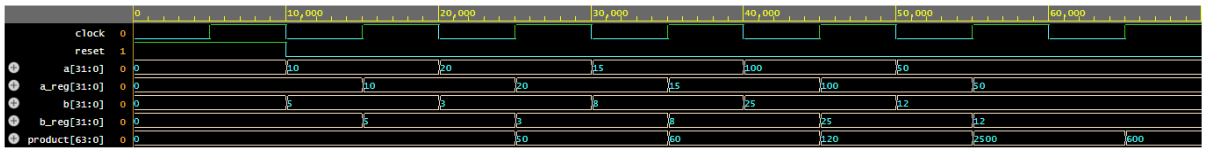
```
module pipelined_multiplier(
    input  logic        clock,
    input  logic        reset,
    input  logic [31:0] a,
    input  logic [31:0] b,
    output logic [63:0] product
);

logic [31:0] a_reg;
logic [31:0] b_reg;

always_ff @(posedge clock or posedge reset) begin
    if (reset) begin
        a_reg <= 32'd0;
        b_reg <= 32'd0;
    end
    else begin
        a_reg <= a;
        b_reg <= b;
    end
end

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        product <= 64'd0;
    else
        product <= a_reg * b_reg;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_pipelined_multiplier;

logic clock;
logic reset;
logic [31:0] a;
logic [31:0] b;
logic [63:0] product;

pipelined_multiplier dut(
    .clock(clock),
    .reset(reset),
    .a(a),
    .b(b),
    .product(product)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    a = 0;
    b = 0;

    #10;
    reset = 0;

    a = 10;     b = 5;
    #10;

    a = 20;     b = 3;
    #10;

    a = 15;     b = 8;
    #10;

    a = 100;    b = 25;
    #10;

    a = 50;     b = 12;
    #20;
```

```

    $finish;
end

initial begin
    $dumpfile("pipelined_multiplier.vcd");
    $dumpvars(0, tb_pipelined_multiplier);
end

endmodule
```

Output:



Mavia Ahmed

Task 18:

Objective:

Implement a **32-bit ALU** in System Verilog.

Design Code:

```
module alu_32bit(
    input  logic [31:0] A,
    input  logic [31:0] B,
    input  logic [2:0]  opcode,
    output logic [31:0] result
);

always_comb begin
    case (opcode)
        3'b000: result = A + B;
        3'b001: result = A - B;
        3'b010: result = A & B;
        3'b011: result = A | B;
        3'b100: result = A ^ B;
        3'b101: result = ~A;
        3'b110: result = A << 1;
        3'b111: result = A >> 1;
        default: result = 32'd0;
    endcase
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_alu_32bit;

logic [31:0] A;
logic [31:0] B;
logic [2:0] opcode;
logic [31:0] result;

alu_32bit dut(
    .A(A),
    .B(B),
    .opcode(opcode),
    .result(result)
);

initial begin

    A = 32'd20;
    B = 32'd10;

    opcode = 3'b000;
    #10;

    opcode = 3'b001;
    #10;

    opcode = 3'b010;
    #10;

    opcode = 3'b011;
    #10;

    opcode = 3'b100;
    #10;

    opcode = 3'b101;
    #10;

    opcode = 3'b110;
    #10;

    opcode = 3'b111;
    #10;

    $finish;
end

initial begin
    $dumpfile("alu_32bit.vcd");
    $dumpvars(0, tb_alu_32bit);
end

endmodule
```

Output:

Mavia Ahmed

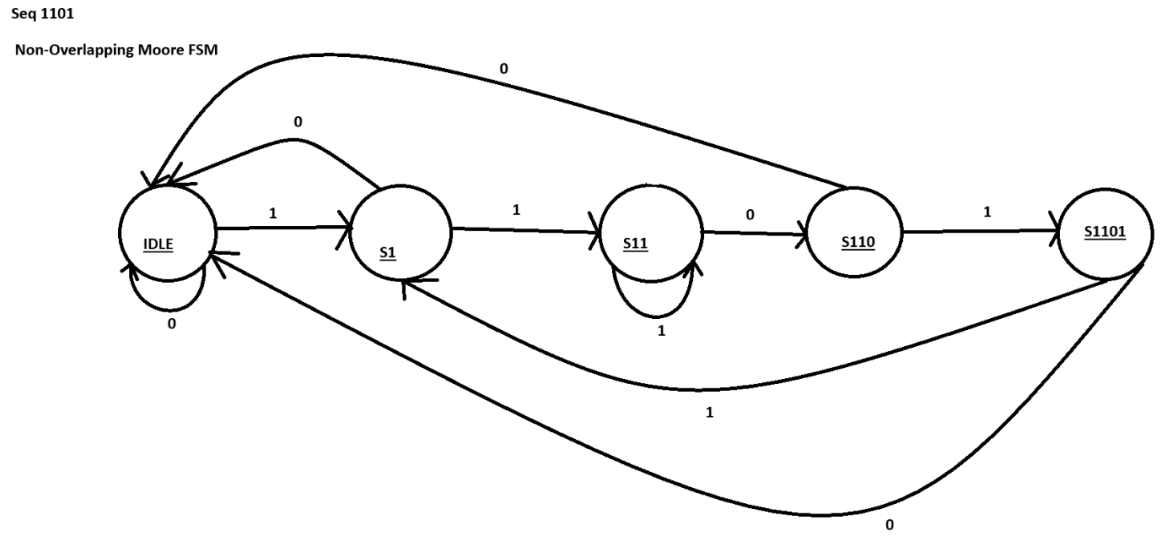
Week No. 3

Task 1:

Objective:

Implement a 1011 **Sequence Detector** using FSM in SystemVerilog.

State Diagram:



Design Code:

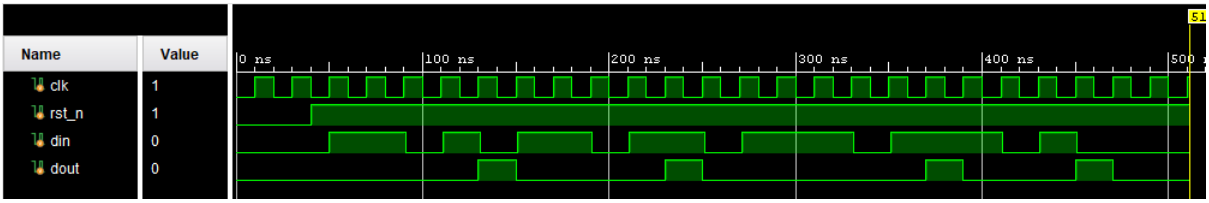
```
module seq_detector_1101_moore (
    input  logic clk,
    input  logic rst_n,
    input  logic din,
    output logic dout
);

    typedef enum logic [2:0] {
        IDLE  = 3'b000,
        S1    = 3'b001,
        S11   = 3'b010,
        S110  = 3'b011,
        S1101 = 3'b100
    } state_t;

    state_t current_state, next_state;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            current_state <= IDLE;
        end else begin
            current_state <= next_state;
        end
    end

    always_comb begin
        next_state = current_state;

        case (current_state)
            IDLE: begin
                if (din) next_state = S1;
                else     next_state = IDLE;
            end

            S1: begin
                if (din) next_state = S11;
                else     next_state = IDLE;
            end

            S11: begin
                if (din) next_state = S11;
                else     next_state = S110;
            end

            S110: begin
                if (din) next_state = S1101;
                else     next_state = IDLE;
            end

            S1101: begin
                if (din) next_state = S1;
                else     next_state = IDLE;
            end
            default: next_state = IDLE;
        endcase
    end

    assign dout = (current_state == S1101);
endmodule
```

Test Bench Code:

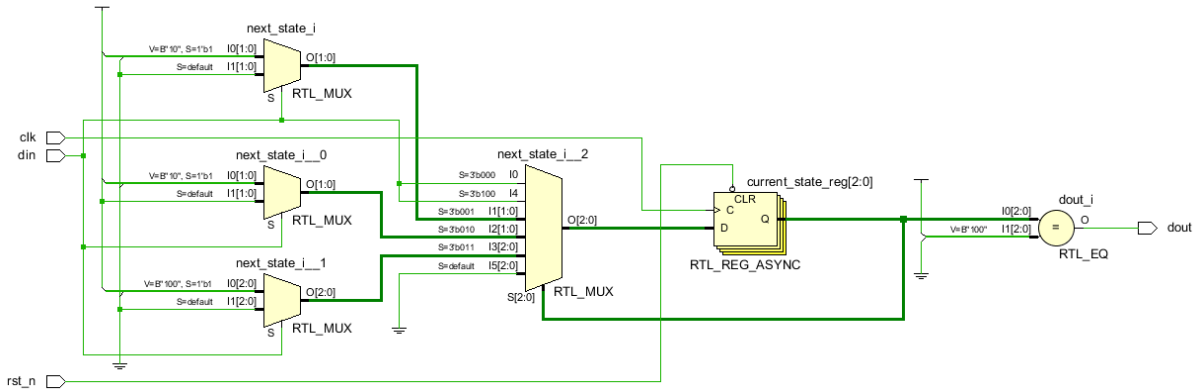
```
`timescale 1ns/1ps

module tb_seq_detector_1101_moore;

    logic clk;
    logic rst_n;
    logic din;
    logic dout;

    seq_detector_1101_moore dut (
        .clk   (clk),
        .rst_n (rst_n),
        .din   (din),
        .dout  (dout)
    );

    always #10 clk = ~clk;

    task send_bit(input logic bit_in);
        begin
            din = bit_in;
            @(posedge clk);
            #1;
        end
    endtask

    initial begin
        clk   = 0;
        rst_n = 0;
        din   = 0;

        #40;
        rst_n = 1;
        @(posedge clk);


        send_bit(1);
        send_bit(1);
        send_bit(0);
        send_bit(1);

        send_bit(0);

        send_bit(1);
        send_bit(1);
        send_bit(0);
        send_bit(1);
        send_bit(1);
        send_bit(0);
        send_bit(1);

        send_bit(1);
        send_bit(1);
        send_bit(0);
        send_bit(1);
        send_bit(1);
        send_bit(1);
        send_bit(0);
        send_bit(1);
```

```
60          send_bit(0);
61
62          #40;
63          $finish;
64      end
65
66    initial begin
67          $dumpfile("dump.vcd");
68          $dumpvars(0, tb_seq_detector_1101_moore);
69    end
70
71  endmodule
```

Simulation Waveforms:



Schematic Diagram:



Mavia Ahmed

Task 2:

Objective:

Implement **Traffic Lights** using FSM in SystemVerilog.

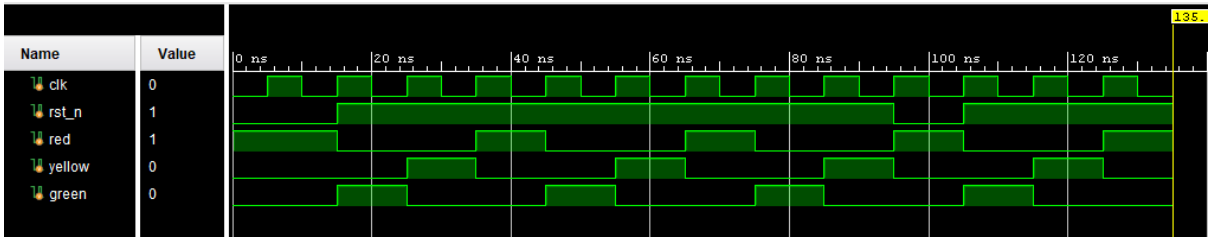
Design Code:

```
module traffic_light_fsm (
    input  logic clk,
    input  logic rst_n,
    output logic red,
    output logic yellow,
    output logic green
);

  typedef enum logic [1:0] {
    S_RED    = 2'b00,
    S_GREEN  = 2'b01,
    S_YELLOW = 2'b10
  } state_t;

  state_t current_state, next_state;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      current_state <= S_RED;
    end else begin
      current_state <= next_state;
    end
  end

  always_comb begin
    case (current_state)
      S_RED:    next_state = S_GREEN;
      S_GREEN:  next_state = S_YELLOW;
      S_YELLOW: next_state = S_RED;
      default:  next_state = S_RED;
    endcase
  end

  always_comb begin
    red    = 1'b0;
    yellow = 1'b0;
    green  = 1'b0;

    case (current_state)
      S_RED:    red    = 1'b1;
      S_GREEN:  green  = 1'b1;
      S_YELLOW: yellow = 1'b1;
    endcase
  end

endmodule
```

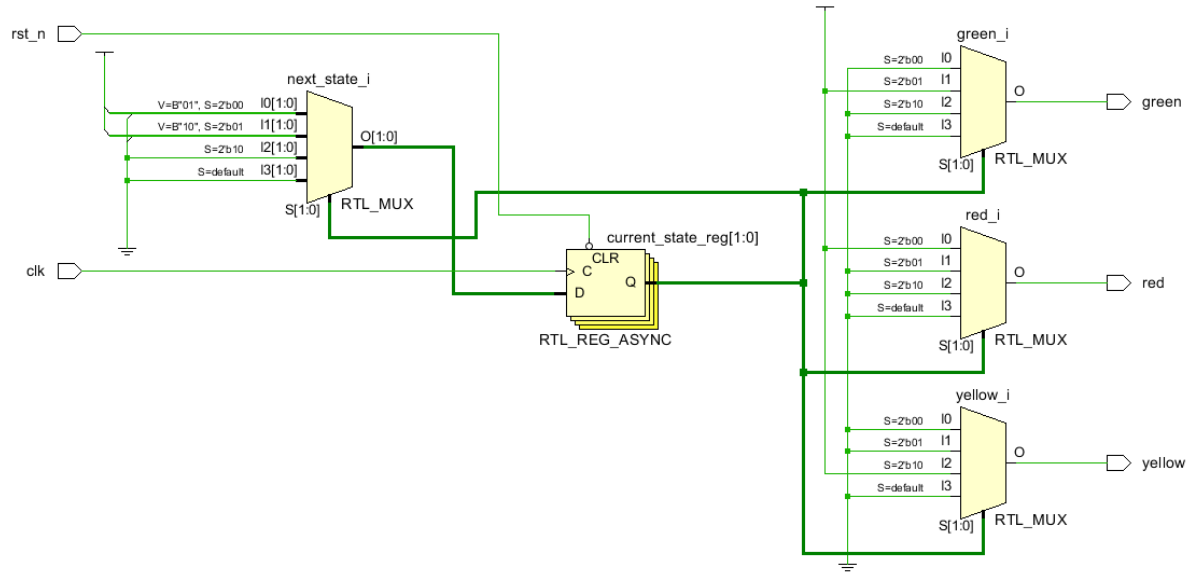
Test Bench Code:

```
`timescale 1ns/1ps

module tb_traffic_light_fsm;

  logic clk;
  logic rst_n;
  logic red;
  logic yellow;
  logic green;

  traffic_light_fsm dut (
    .clk(clk),
    .rst_n(rst_n),
    .red(red),
    .yellow(yellow),
    .green(green)
  );

  always begin
    #5 clk = ~clk;
  end

  initial begin
    clk = 0;
    rst_n = 0;

    #15 rst_n = 1;

    #80;

    rst_n = 0;
    #10;
    rst_n = 1;
    #30;

    $finish;
  end

initial begin
    $dumpfile("traffic_light_tb.vcd");
    $dumpvars(0, tb_traffic_light_fsm);
  end

endmodule
```

Simulation Waveforms:



Schematic Diagram:



Mavia Ahmed

Task 3:

Objective:

Implement a **Vending Machine** using FSM in SystemVerilog.

Design Code:

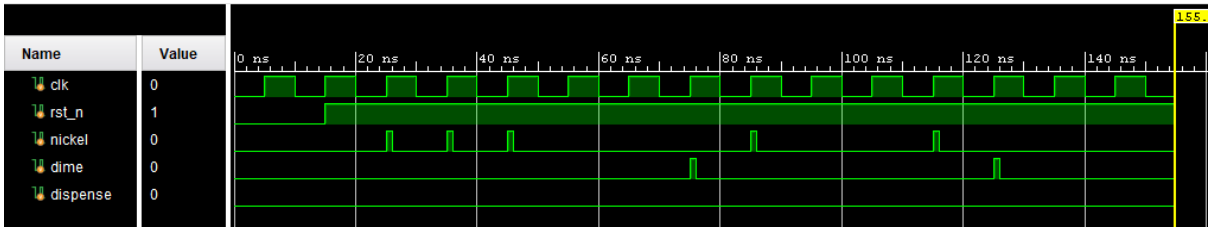
```
module vending_machine (
    input  logic clk,
    input  logic rst_n,
    input  logic nickel,
    input  logic dime,
    output logic dispense
);

  typedef enum logic [1:0] {
    S_0c  = 2'b00,
    S_5c  = 2'b01,
    S_10c = 2'b10,
    S_15c = 2'b11
  } state_t;

  state_t current_state, next_state;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      current_state <= S_0c;
    end else begin
      current_state <= next_state;
    end
  end

  always_comb begin
    next_state = current_state;

    case (current_state)
      S_0c: begin
        if (nickel)      next_state = S_5c;
        else if (dime)   next_state = S_10c;
      end

      S_5c: begin
        if (nickel)      next_state = S_10c;
        else if (dime)   next_state = S_15c;
      end

      S_10c: begin
        if (nickel || dime) next_state = S_15c;
      end

      S_15c: begin
        next_state = S_0c;
      end

      default: next_state = S_0c;
    endcase
  end

  assign dispense = (current_state == S_15c);

endmodule
```

Test Bench Code:

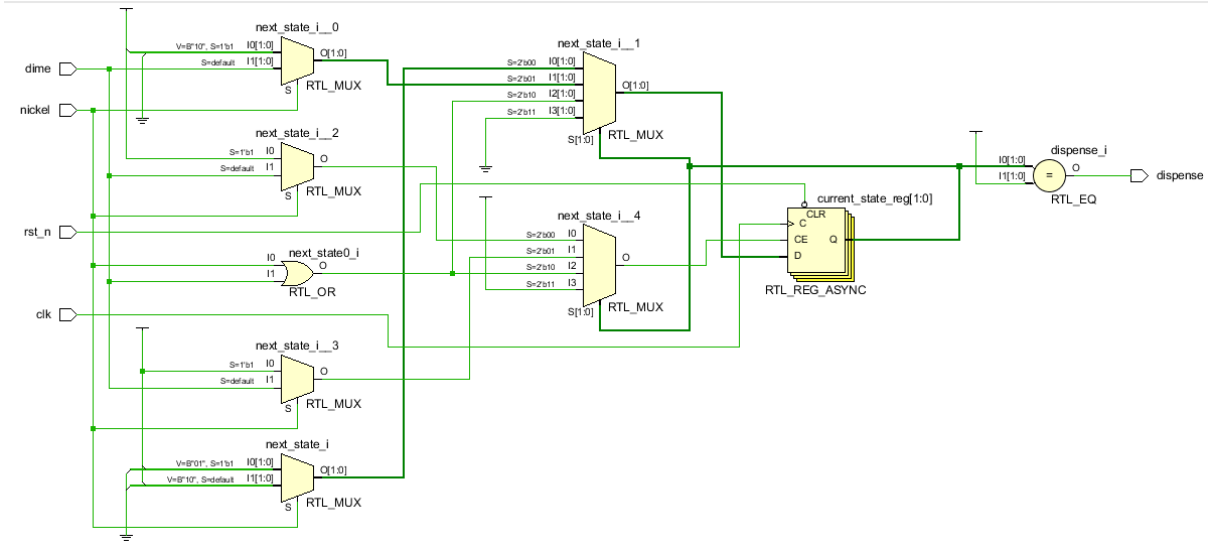
```
`timescale 1ns/1ps

module tb_vending_machine;

  logic clk;
  logic rst_n;
  logic nickel;
  logic dime;
  logic dispense;

  vending_machine dut (
    .clk(clk),
    .rst_n(rst_n),
    .nickel(nickel),
    .dime(dime),
    .dispense(dispense)
  );

  always begin
    #5 clk = ~clk;
  end

  task insert_nickel();
    @(posedge clk);
    nickel = 1;
    #1;
    nickel = 0;
  endtask

  task insert_dime();
    @(posedge clk);
    dime = 1;
    #1;
    dime = 0;
  endtask

  initial begin
    clk = 0;
    rst_n = 0;
    nickel = 0;
    dime = 0;

    #15 rst_n = 1;
    #5;

    insert_nickel();
    insert_nickel();
    insert_nickel();
    @(posedge clk);

    #20;

    insert_dime();
    insert_nickel();
    @(posedge clk);

    #20;
```

```
58
59      insert_nickel();
60      insert_dime();
61      @(posedge clk);
62
63      #20;
64      $finish;
65    end
66
67    initial begin
68      $dumpfile("vending_machine_tb.vcd");
69      $dumpvars(0, tb_vending_machine);
70    end
71
72  endmodule
```

Simulation Waveforms:



Schematic Diagram:



Mavia Ahmed

Task 4:

Objective:

Implement an 8-bit **FIFO Buffer** in SystemVerilog.

Design Code:

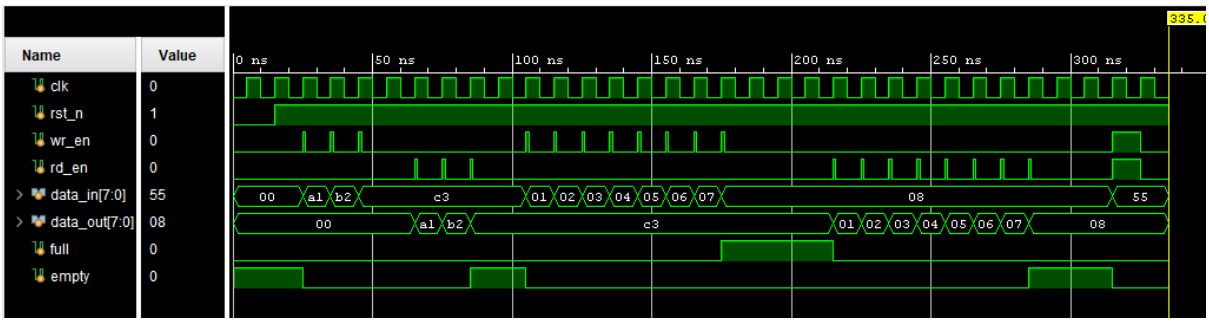
```
module fifo_8bit (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       wr_en,
    input  logic       rd_en,
    input  logic [7:0] data_in,
    output logic [7:0] data_out,
    output logic       full,
    output logic       empty
);

  parameter DEPTH = 8;

  logic [7:0] memory [DEPTH-1:0];

  logic [2:0] wr_ptr;
  logic [2:0] rd_ptr;
  logic [3:0] count;

  assign empty = (count == 0);
  assign full  = (count == DEPTH);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      wr_ptr   <= 3'b0;
      rd_ptr   <= 3'b0;
      count    <= 4'b0;
      data_out <= 8'b0;
    end else begin

      if (wr_en && !full) begin
        memory[wr_ptr] <= data_in;
        wr_ptr         <= wr_ptr + 1;
      end

      if (rd_en && !empty) begin
        data_out <= memory[rd_ptr];
        rd_ptr   <= rd_ptr + 1;
      end

      if (wr_en && !full && !(rd_en && !empty)) begin
        count <= count + 1;
      end else if (rd_en && !empty && !(wr_en && !full)) begin
        count <= count - 1;
      end
    end
  end

endmodule
```

Test Bench Code:

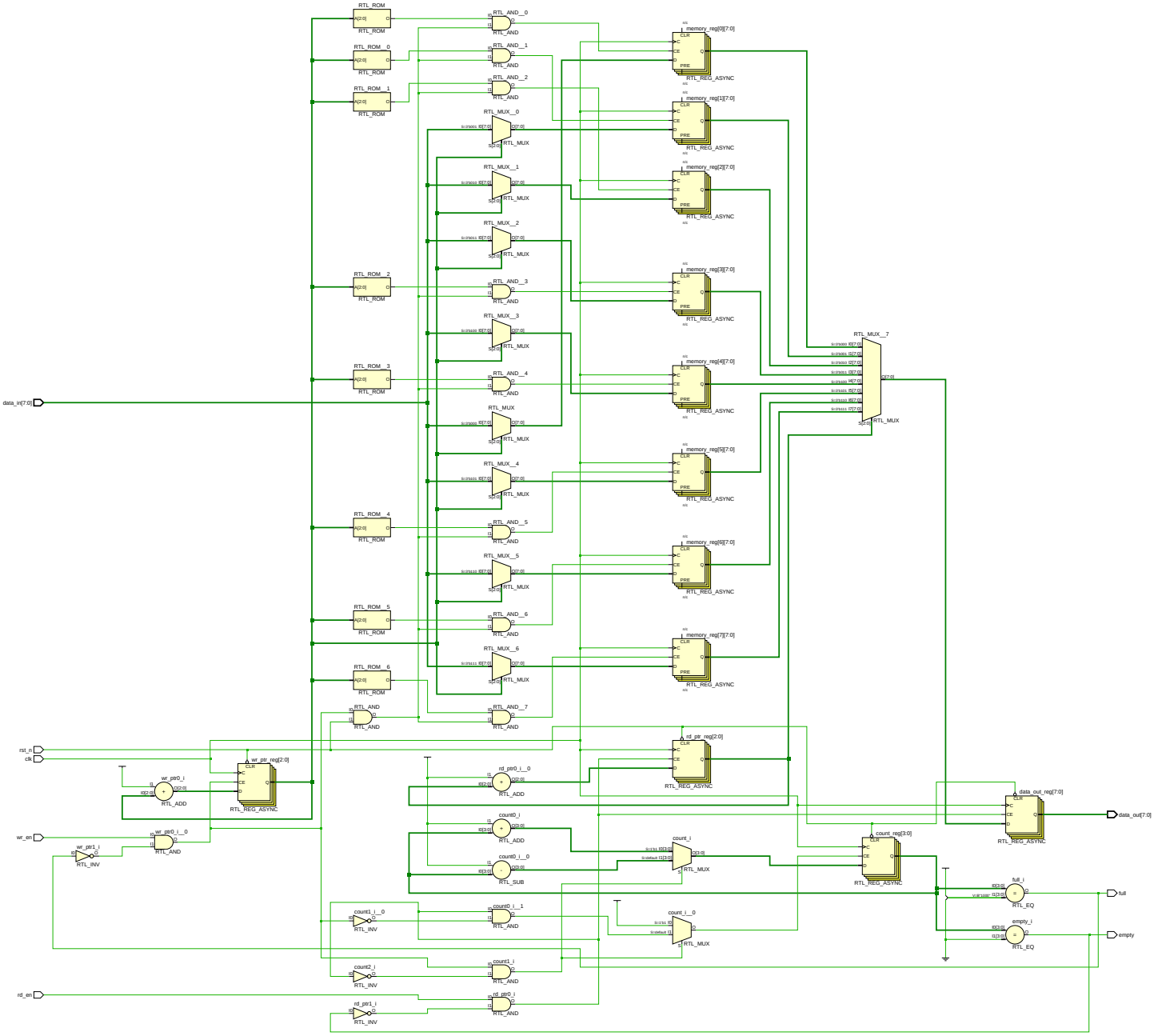
```
`timescale 1ns/1ps

module tb_fifo_8bit;

  logic       clk;
  logic       rst_n;
  logic       wr_en;
  logic       rd_en;
  logic [7:0] data_in;
  logic [7:0] data_out;
  logic       full;
  logic       empty;

  fifo_8bit dut (
    .clk(clk),
    .rst_n(rst_n),
    .wr_en(wr_en),
    .rd_en(rd_en),
    .data_in(data_in),
    .data_out(data_out),
    .full(full),
    .empty(empty)
  );

  always #5 clk = ~clk;

  initial begin
    clk = 0;
    rst_n = 0;
    wr_en = 0;
    rd_en = 0;
    data_in = 8'h0;

    #15 rst_n = 1;
    #5;

    write_data(8'hA1);
    write_data(8'hB2);
    write_data(8'hC3);
    #10;

    read_data();
    read_data();
    read_data();
    #10;

    for (int i = 1; i <= 8; i++) begin
      write_data(i);
    end
    #10;

    write_data(8'hFF);
    #10;

    for (int i = 1; i <= 8; i++) begin
      read_data();
    end
    #10;
```

```
59
60      read_data();
61      #10;
62
63      @(posedge clk);
64      wr_en = 1;
65      rd_en = 1;
66      data_in = 8'h55;
67      #10;
68      wr_en = 0;
69      rd_en = 0;
70      #10;
71
72      $finish;
73    end
74
75    task write_data(input logic [7:0] val);
76      @(posedge clk);
77      if (!full) begin
78        wr_en = 1;
79        data_in = val;
80      end else begin
81      end
82      #1;
83      wr_en = 0;
84    endtask
85
86    task read_data();
87      @(posedge clk);
88      if (!empty) begin
89        rd_en = 1;
90        #1;
91      end else begin
92      end
93      rd_en = 0;
94    endtask
95
96    initial begin
97      $dumpfile("fifo_8bit_tb.vcd");
98      $dumpvars(0, tb_fifo_8bit);
99    end
100
101 endmodule
```

Simulation Waveforms:



Schematic Diagram:



Mavia Ahmed

Task 5:

Objective:

To design and verify a PS/2 keyboard receiver in SystemVerilog capable of receiving keyboard scan codes and detecting the keys A, B, and C.

Design Code:

```
module ps2_keyboard_receiver(

    input logic clk,
    input logic rst,
    input logic ps2_clk,
    input logic ps2_data,

    output logic A,
    output logic B,
    output logic C,
    output logic data_valid
);

logic ps2_clk_d;

logic [10:0] shift_reg;
logic [3:0] bit_count;

logic [10:0] next_shift;

always_ff @(posedge clk or posedge rst)
begin
    if(rst)
    begin
        ps2_clk_d <= 1;

        shift_reg <= 0;
        bit_count <= 0;

        A <= 0;
        B <= 0;
        C <= 0;

        data_valid <= 0;
    end
    else
    begin
        ps2_clk_d <= ps2_clk;

        A <= 0;
        B <= 0;
        C <= 0;
        data_valid <= 0;

```

```
45          if(ps2_clk_d && !ps2_clk)
46          begin
47              next_shift = {ps2_data,shift_reg[10:1]};
48
49              shift_reg <= next_shift;
50
51              if(bit_count == 10)
52              begin
53                  bit_count <= 0;
54
55                  data_valid <= 1;
56
57                  case(next_shift[8:1])
58
59                      8'h1C: A <= 1;
60                      8'h32: B <= 1;
61                      8'h21: C <= 1;
62
63                  endcase
64              end
65              else
66              begin
67                  bit_count <= bit_count + 1;
68              end
69          end
70      end
71  end
72
73  endmodule
```

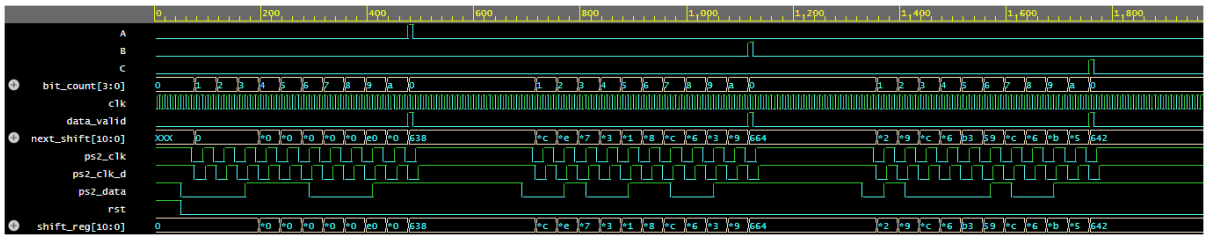
Testbench Code:

```
module tb_ps2_keyboard_receiver;

logic clk;
logic rst;

logic ps2_clk;
logic ps2_data;

logic A;
logic B;
logic C;
logic data_valid;

ps2_keyboard_receiver DUT(
    .clk(clk),
    .rst(rst),
    .ps2_clk(ps2_clk),
    .ps2_data(ps2_data),
    .A(A),
    .B(B),
    .C(C),
    .data_valid(data_valid)
);

initial begin
    $dumpfile("ps2.vcd");
    $dumpvars(0,tb_ps2_keyboard_receiver);
end

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

task send_bit(input logic b);
begin
    ps2_data = b;

    #20;
    ps2_clk = 0;

    #20;
    ps2_clk = 1;
end
```

```
endtask

task send_code(input [7:0] code);
integer i;
begin

    // Start bit
    send_bit(0);

    // Data bits (LSB first)
    for(i=0;i<8;i=i+1)
        send_bit(code[i]);

    // Parity bit
    send_bit(1);

    // Stop bit
    send_bit(1);

end
endtask

initial begin

    rst = 1;
    ps2_clk = 1;
    ps2_data = 1;

    #50;
    rst = 0;

    // A = 1C
    send_code(8'h1C);

    #200;

    // B = 32
    send_code(8'h32);

    #200;

    // C = 21
    send_code(8'h21);


    #200;

    $finish;

end

endmodule
```

Mavia Ahmed

Simulation Waveforms:



Task 6:

Objective:

To design and verify an APB-based RAM module capable of performing read and write operations using the APB protocol.

Design Code:

```
module apb_ram
#(
    parameter ADDR_WIDTH = 4,
    parameter DATA_WIDTH = 32
)
(
    input  logic                    PCLK,
    input  logic                    PRESETn,

    input  logic                    PSEL,
    input  logic                    PENABLE,
    input  logic                    PWRITE,

    input  logic [ADDR_WIDTH-1:0]   PADDR,
    input  logic [DATA_WIDTH-1:0]   PWDATA,

    output logic [DATA_WIDTH-1:0]   PRDATA,
    output logic                    PREADY
);

logic [DATA_WIDTH-1:0] mem [0:(1<<ADDR_WIDTH)-1];

assign PREADY = 1'b1;

integer i;

always_ff @(posedge PCLK or negedge PRESETn)
begin
    if(!PRESETn)
    begin
        PRDATA <= 0;

        for(i=0;i<(1<<ADDR_WIDTH);i=i+1)
            mem[i] <= 0;
    end
    else
    begin
        if(PSEL && PENABLE)
        begin
            if(PWRITE)
            begin
                mem[PADDR] <= PWDATA;
            end
            else

            begin
                PRDATA <= mem[PADDR];
            end
        end
    end
end

endmodule
```

Testbench Code:

```
module tb_apb_ram;

logic PCLK;
logic PRESETn;

logic PSEL;
logic PENABLE;
logic PWRITE;

logic [3:0] PADDR;
logic [31:0] PWDATA;

logic [31:0] PRDATA;
logic PREADY;

apb_ram DUT(
    .PCLK(PCLK),
    .PRESETn(PRESETn),
    .PSEL(PSEL),
    .PENABLE(PENABLE),
    .PWRITE(PWRITE),
    .PADDR(PADDR),
    .PWDATA(PWDATA),
    .PRDATA(PRDATA),
    .PREADY(PREADY)
);

initial
begin
    $dumpfile("apb_ram.vcd");
    $dumpvars(0,tb_apb_ram);
end

initial
begin
    PCLK = 0;
    forever #5 PCLK = ~PCLK;
end

initial
begin

    PRESETn = 0;

```

```
45      PSEL = 0;
46      PENABLE = 0;
47      PWRITE = 0;
48
49      PADDR = 0;
50      PWDATA = 0;
51
52      #20;
53      PRESETn = 1;
54
55      // Write 1111AAAA to Address 2
56      @(posedge PCLK);
57
58      PSEL    = 1;
59      PENABLE = 0;
60      PWRITE  = 1;
61      PADDR   = 4'd2;
62      PWDATA  = 32'h1111AAAA;
63
64      @(posedge PCLK);
65
66      PENABLE = 1;
67
68      @(posedge PCLK);
69
70      PSEL = 0;
71      PENABLE = 0;
72
73      // Write DEADBEEF to Address 5
74      @(posedge PCLK);
75
76      PSEL    = 1;
77      PENABLE = 0;
78      PWRITE  = 1;
79      PADDR   = 4'd5;
80      PWDATA  = 32'hDEADBEEF;
81
82      @(posedge PCLK);
83
84      PENABLE = 1;
85
86      @(posedge PCLK);
87
```

```
88      PSEL = 0;
89      PENABLE = 0;
90
91      // Read Address 2
92      @(posedge PCLK);
93
94      PSEL = 1;
95      PENABLE = 0;
96      PWRITE = 0;
97      PADDR = 4'd2;
98
99      @(posedge PCLK);
100
101     PENABLE = 1;
102
103     @(posedge PCLK);
104
105     PSEL = 0;
106     PENABLE = 0;
107
108     // Read Address 5
109     @(posedge PCLK);
110
111     PSEL = 1;
112     PENABLE = 0;
113     PWRITE = 0;
114     PADDR = 4'd5;
115
116     @(posedge PCLK);
117
118     PENABLE = 1;
119
120     @(posedge PCLK);
121
122     PSEL = 0;
123     PENABLE = 0;
124
125     #50;
126
127     $finish;
128
129 end
130
131 endmodule
```

Simulation Waveforms: